The Most Efficient Protocol for PAKE:

What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Security Analysis of Basic OEKE

2.1 The Simulator

2.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Extracting password guess from malicious P. When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P', the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding $\mathsf{TestPwd}$ command to $\mathcal{F}_{\mathrm{PAKE}}$. There are two types of attack, which need to be addressed separately:

1. *"Normal" online attack:* $\mathcal{A}$ executes P's algorithm on a password guess pw^*. That is, $\mathcal{A}$ samples public key pk^* and randomness r^*, computes

   $$R^* := H_1(\mathsf{pw}^*, r^*), \qquad T^* := pk^* \cdot R^*,$$
   $$t^* := H_2(\mathsf{pw}^*, T^*), \qquad s^* := t^* \oplus r^*,$$

   and sends (s^*, T^*) to P'.
   It takes a while to realize how to extract pw^* from (s^*, T^*). First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the "actual" password guess. This can be done via the following. For each such (pw^*, t^*) pair, $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\mathsf{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\mathsf{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the "actual" password guess pw^*. (This special $H_1(\mathsf{pw}^*, r^*)$ query is called the "anchor query" in [MRR20, JRX25].)

2. *Related key attack:* This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P. $\mathcal{A}$ skips sampling pk^* or r^*; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$t^* := H_2(\mathsf{pw}^*, T), \qquad (t^*)' := H_2(\mathsf{pw}^*, T^*),$$
$$\delta := t^* \oplus (t')^*, \qquad s^* := s \oplus \delta,$$

and sends (s^*, T^*) to P'.
Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*), such that

$$s \oplus s^* = H_2(\mathsf{pw}, T) \oplus H_2(\mathsf{pw}, T^*),$$

which allows $\mathcal{A}$ to pick T^* and "reverse engineer" the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\mathsf{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.
To extract pw^* from (s^*, T^*), $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which one satisfies

$$H_2(\mathsf{pw}^*, T) \oplus H_2(\mathsf{pw}^*, T^*) = s \oplus s^*.$$

With overwhelming probability, at most one pw^* will satisfy this equation.

References

[ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Report 2025/231, 2025.

[JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.